

The Digital Ear - Stage 2: GHOSTFM



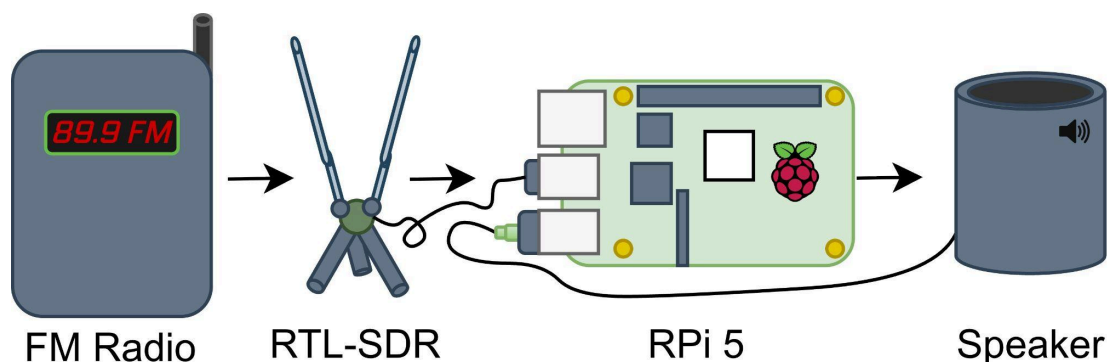
Updated architecture walkthrough: [The Digital Ear / GhostFM](#)
GhostFM Live Demo (3min): <https://youtu.be/CaB2kfwomGc>

1. Architecture

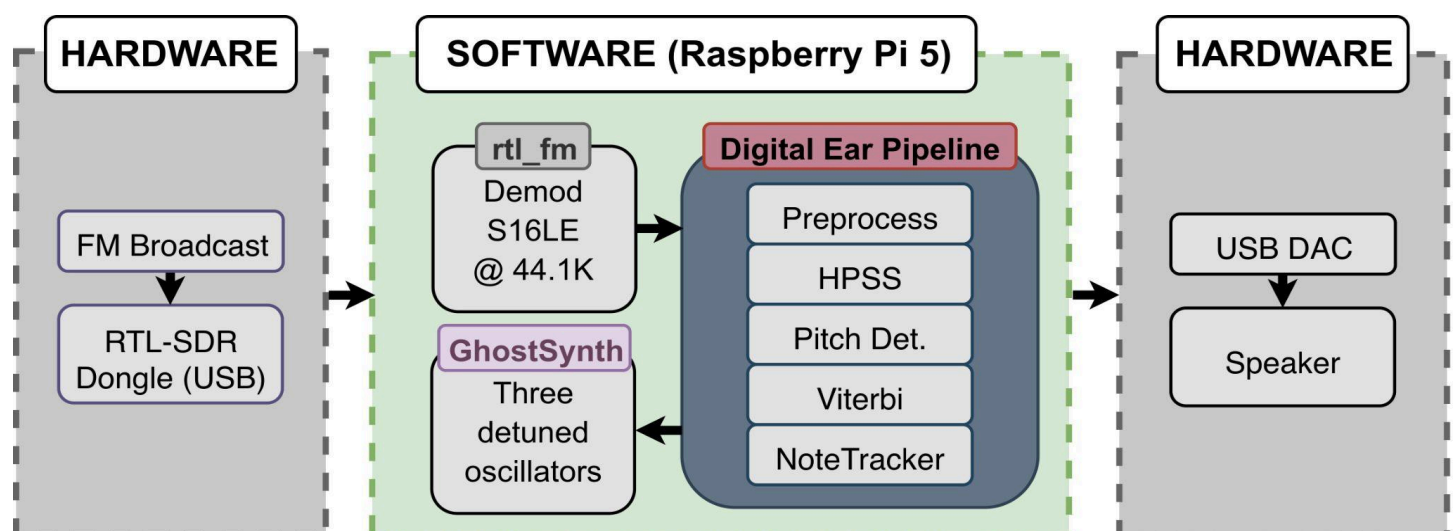
1.1 System Overview

Ghost FM is a standalone embedded device that receives over-the-air FM radio via software-defined radio (SDR), extracts the dominant melody in real time using the Digital Ear pipeline, and re-synthesizes it as a ghostly, detuned sine tone through a USB audio interface. The entire system runs headless on a Raspberry Pi 5, requiring no external monitor or keyboard after initial setup. To support this, a 1.3" LCD HAT provides a compact status display along with basic user input (joystick and buttons) for tuning and control.

1.2 GhostFM Hardware Overview



1.3 High Level Signal Flow



2. Tradeoffs (Latency vs. Frequency Resolution)

Pitch detection has a fundamental trade-off: improving frequency precision requires longer observation windows, which directly increases latency. The system parameters below reflect how this trade-off was balanced for real-time performance on noisy FM input.

2.1 Frequency Resolution

Parameter	Value	Effect
Window Length	2048 samples (~46ms)	Sets base frequency resolution (~21.5Hz)
FFT size	4096 (zero-padded)	Interpolates spectrum to ~10.8 Hz bin spacing
Hop size	512 samples (~11.6 ms)	~86 updates/sec for responsive onset detection
IF refinement	Phase-based	Improves pitch estimate to sub-bin accuracy

The core limitation comes from the 2048-sample window, which fixes the true frequency resolution at about 21.5 Hz. Zero-padding to 4096 does not add new information, but it gives a denser frequency grid that makes peak estimation more stable.

To get around the coarse bin spacing, I use phase-based instantaneous frequency refinement. This lets me estimate pitch between FFT bins, so in practice the system achieves much finer resolution than the raw FFT would suggest.

The result is a balance: short enough windows to stay responsive, but enough frequency precision to track melody accurately.

2.2 Latency Budget: Where Time Is Spent

Stage	Latency	Purpose
HPSS (kernel = 31)	~175ms	Removes percussive/transient energy before pitch tracking
Viterbi (lag=50)	~580 ms	Stabilizes pitch over time, resolves octave errors
NoteTracker (median=15)	~17 ms	Smooths short-term jitter
GhostSynth (oscillator)	< 1 ms	Immediate audio generation
Total	~800 ms	

Most of the latency comes from enforcing stability on a noisy FM signal.

The largest contributor is the Viterbi decoder. It waits ~50 frames before committing to a pitch, which gives enough temporal context to avoid octave jumps and random errors. A shorter lag reduces latency but quickly becomes unstable, especially on weaker stations.

HPSS is the second major cost. Without it, percussion and transients from the radio create false note detections. The added delay is worth it because it significantly cleans up the input before pitch estimation.

The NoteTracker adds a small amount of smoothing to remove frame-to-frame jitter, but its contribution to latency is minor.

Overall latency is around 800 ms. This was an intentional choice. The goal of the system is to produce a clean, musically stable melody, not to perfectly align with the radio signal in time. In practice, a slightly delayed but stable output sounds much more convincing than a low-latency signal that jumps between incorrect notes.

3. Bill Of Materials

All components fit within the \$250 budget at \$222 total.

Part Name	Link	Qty	Cost
Raspberry Pi 5 - 4GB	Central Computer	1	\$85.00
Raspberry Pi 27W USB-C Power Supply (Black)	Digikey	1	\$12.00
SABRENT Aluminum USB External Stereo Sound Adapter	Amazon	1	\$9.00
Raspberry Pi Active Fan	Amazon	1	\$10.00
RTL-SDR Blog V4 Radio with Dipole Antenna Kit	Amazon	1	\$50.00
Waveshare 1.3inch IPS LCD Display HAT	Amazon	1	\$17.00
13.5mm GPIO and PoE Pin Header Height Extender	Amazon	1	\$7.00
BOESPATR Mini Portable FM Radio	Amazon	1	\$17.00
LENRUE Portable Bluetooth Speaker	Amazon	1	\$10.00
Aux Cord	Amazon	1	\$5.00
TOTAL:			\$222.00